

Kvantizacija neuronskih mreža korišćenjem ONNX formata

Nemanja Ljubičić, E9 8/2025
Fakultet tehničkih nauka
Univerzitet u Novom Sadu
Novi Sad, Srbija
nemanjaljubcic2@gmail.com

Apstrakt- U radu je prikazan kompletan tok kvantizacije konvolucione neuronske mreže LeNet-5, istrenirane za klasifikaciju rukom pisanih cifara iz skupa MNIST i izvezene u ONNX format. Nakon provere ispravnosti izvoza i optimizacije grafa, implementirane su i međusobno upoređene tri varijante kvantizacije koje postepeno obuhvataju sve veći deo modela (samo težine, težine i bias, kao i težine, bias i aktivacije), svaka u INT8 i INT16 preciznosti. Za svaku varijantu izmerene su tačnost na test skupu, veličina fajla modela i brzina zaključivanja na CPU. Date su perspektive za niže bitske preciznosti, INT4 i INT2.

Ključne reči - kvantizacija, ONNX, konvoluciona neuronska mreža, CNN, LeNet-5, MNIST, INT8, INT16

I. UVOD

Modeli dubokog učenja se najčešće treniraju i čuvaju u tipu podataka FP32. Za primenu na uređajima sa ograničenim resursima, javlja se potreba za smanjenjem veličine modela sa što manjim gubitkom tačnosti.

Kvantizacija je jedna od tehnika kompresije modela, gde se brojevi u pokretnom zarezu preslikavaju u celobrojni tip (najčešće INT8 ili INT16), čime se smanjuje veličina zapisa parametara i, pod određenim uslovima, ubrzava izvršavanje na hardveru koji podržava celobrojnu aritmetiku.

Cilj ovog rada je da se na konkretnom primeru, sprovede i uporedi nekoliko varijanti kvantizacije koje postepeno obuhvataju sve veći deo modela, i da se izmeri njihov uticaj na tačnost, veličinu modela i brzinu zaključivanja.

Rad je organizovan tako da su u odeljku II ukratko opisani osnovni pojmovi i tipovi kvantizacije korišćeni u nastavku rada. Odeljak III opisuje arhitekturu mreže i postupak obuke na skupu MNIST. Odeljak IV opisuje ONNX format i izvoz istreniranog modela u taj format i proveru ispravnosti tog izvoza, kao i optimizaciju grafa. Odeljak V detaljno opisuje tri implementirane varijante kvantizacije i strukturu dobijenih grafova. Odeljak VI poredi dobijene modele po tačnosti, veličini i brzini izvršavanja. U odeljku VII se razmatraju perspektive za niže bitske preciznosti, dok odeljak VIII sumira zaključke rada.

II. TIPOVI KVANTIZACIJE

Kvantizacija je postupak preslikavanja realnog opsega vrednosti u ograničen skup celih brojeva pomoću faktora skaliranja i referentne nulte vrednosti.

Postoji nekoliko podela kvantizacije, na osnovu toga kada se obavlja, kako se preslikava polazni opseg, bitske preciznosti i granularnosti. Za svaku podelu biće navedeno koji tip je korišćen u ovom radu. Podela i preporuke su preuzete iz zvanične ONNX dokumentacije [1]

Simetrična i asimetrična kvantizacija: kod simetrične kvantizacije referentna nulta vrednost je fiksirana na 0, a opseg celih brojeva je simetričan oko nule, što je korišćeno za težine mreže. Kod asimetrične kvantizacije referentna vrednost može biti pomerena, što bolje odgovara veličinama koje nisu simetrične oko nule, što je ovde korišćeno za aktivacije.

Po tenzoru naspram po kanalu: kod kvantizacije po tenzoru, čitav tenzor deli jedan zajednički faktor skaliranja, dok kod kvantizacije po kanalu, svaki izlazni kanal dobija sopstveni faktor skaliranja. Za težine je korišćen pristup po kanalu, dok je za biase i aktivacije korišćena kvantizacija po tenzoru.

Što se tiče bitskih preciznosti, u radu su korišćene INT8 i INT16 reprezentacije, dok će se niže preciznosti teorijski obraditi.

Kvantizacija nakon obuke naspram kvantizacije svesne obuke: u ovom radu korišćena je kvantizacija nakon obuke (*post-training quantization*), gde se faktori skaliranja određuju nad već istreniranim modelom, bez dodatnog ponovnog treniranja. Kod varijante koja obuhvata i aktivacije, ovaj postupak zahteva prethodni korak kalibracije (statička kvantizacija), u kom se kroz mrežu propusti reprezentativan uzorak podataka kako bi se procenio opseg vrednosti aktivacija na svakom mestu u grafu.

III. ARHITEKTURA MREŽE I OBUKA

Za klasifikaciju rukom pisanih cifara iz skupa MNIST [2] korišćena je konvoluciona neuronska mreža LeNet-5, koju su 1998. godine predložili Yann LeCun i saradnici za problem prepoznavanja rukom pisanih karaktera [3].

Mreža se sastoji od naizmeničnih konvolucionih i *pooling* slojeva, nakon kojih sledi niz potpuno povezanih (*fully connected*) slojeva koji vrše konačnu klasifikaciju.

Ulaz u mrežu predstavlja siva slika dimenzija 28×28 piksela, koja je dopunjena *zero padding-om* na dimenzije 32×32 , u skladu sa originalnom LeNet-5 arhitekturom.

Prvi konvolucionni sloj primenjuje 6 filtera dimenzija 5×5 sa *padding-om* 2 na ulaznu sliku, čime se dobija 6 mapa obeležja dimenzija 28×28 . Nakon konvolucije primenjuje se ReLU aktivaciona funkcija, a zatim *max pooling* 2×2 , koji smanjuje prostorne dimenzije na 14×14 .

Drugi konvolucionni sloj primenjuje 16 filtera dimenzija 5×5 (bez *padding-a*) nad izlazom prethodnog sloja, čime se dobija 16 mapa obeležja dimenzija 10×10 . Ponovo se primenjuje ReLU, a zatim *max pooling* 2×2 , koji dimenzije svodi na 5×5 .

Na ovom mestu izlaz konvolucionog dela mreže, dimenzija $16 \times 5 \times 5 = 400$ vrednosti, se transformiše (*flatten*)

u jednodimenzionim vektor i prosleđuje potpuno povezanim slojevima.

Prvi potpuno povezani sloj preslikava 400 ulaznih vrednosti u 120 izlaznih neurona, sa ReLU aktivacijom.

Drugi potpuno povezani sloj preslikava 120 u 84 neurona, takođe sa ReLU aktivacijom.

Izlazni sloj preslikava 84 vrednosti u 10 izlaznih neurona, po jedan za svaku od 10 klasa (cifre 0–9). Na ovom sloju se ne primenjuje aktivaciona funkcija unutar mreže, jer se softmax implicitno primenjuje kroz funkciju gubitka (*CrossEntropyLoss*) prilikom obuke.

U odnosu na originalnu LeNet-5 arhitekturu iz 1998. godine, u ovoj implementaciji izvršena su dva odstupanja: umesto sigmoidalne/tanh aktivacije i *average pooling*-a korišćeni su ReLU aktivaciona funkcija i *max pooling*, što predstavlja uobičajenu modernu verziju arhitekture [4].

A. Skup podataka i priprema

Nad ulaznim slikama primenjena je normalizacija sa srednjom vrednošću 0.1307 i standardnom devijacijom 0.3081, što su unapred izračunate statistike celog MNIST trening skupa, preuzete iz zvaničnog PyTorch primera za MNIST [5].

Originalni trening skup od 60.000 slika podeljen je na trening i validacioni skup u razmeri 55.000 : 5.000, dok je test skup od 10.000 slika zadržan u originalnom obliku.

Podaci su učitavani sa sledećim veličinama *batch*-a: 64 za trening skup (uz mešanje redosleda primera u svakoj epohi), i 256 za validacioni i test skup.

B. Postupak obuke

Obuka modela je izvedena korišćenjem sledećih hiperparametara i strategija:

Kao funkcija gubitka korišćena je unakrsna entropija (*CrossEntropyLoss*).

Za optimizaciju parametara mreže korišćen je Adam optimizator sa stopom učenja od 0.001 ($1e-3$).

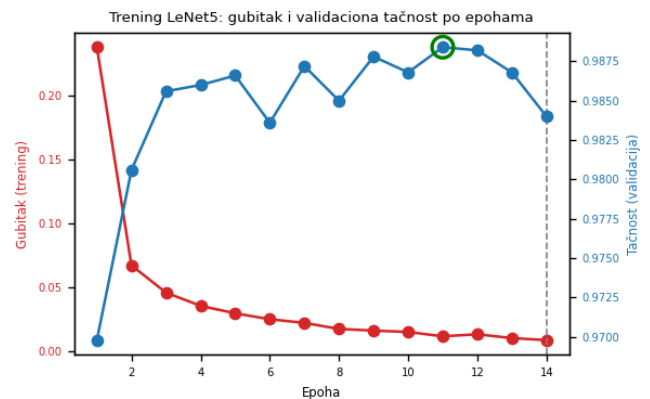
Postavljen je maksimalan broj epoha od 30, uz mehanizam ranog zaustavljanja sa čekanjem od 3 epohe. Nakon svake epohe model se evaluira na validacionom skupu, i ukoliko se validaciona tačnost ne poboljša u odnosu na dotadašnji najbolji rezultat tri epohe zaredom, obuka se prekida pre dostizanja maksimalnog broja epoha.

Tokom obuke se nakon svake epohe upoređuje tekuća validaciona tačnost sa najboljom dosad postignutom. Ukoliko je tekuća tačnost veća, parametri modela se snimaju na disk. Nakon završetka obuke u model se učitavaju sačuvani parametri iz epohe sa najboljom validacionom tačnošću, čime se za dalju evaluaciju koristi najbolja verzija modela.

Za svaku epohu beleži se i ispisuje prosečna vrednost funkcije gubitka na trening skupu, kao i tačnost na validacionom skupu.

C. Rezultati obuke

Trening je zaustavljen ranim zaustavljanjem na 14. epohi (od maksimalno 30 predviđenih), pošto se validaciona tačnost tri epohe zaredom nije poboljšala u odnosu na dotadašnji najbolji rezultat. Kretanje gubitka na trening skupu i tačnosti na validacionom skupu kroz obuku prikazano je na Slici III.1.



Slika III.1 Tok obuke modela

Za dalju evaluaciju korišćene su težine iz epohe 11, u kojoj je zabeležena najveća validaciona tačnost (0.9884). Na test skupu ta verzija modela ostvarila je tačnost od 98.91%.

IV. ONNX FORMAT

ONNX format omogućava da se model čuva kao računski graf koji sadrži sve što je potrebno za kasnije rekreiranje mreže: operacije koje se izvode, tok podataka kroz njih, kao i naučene težine i bias-e.

Sastoji se od ulaza (tip i oblik tenzora koji mreža prima), čvorova (svaki čvor predstavlja jednu operaciju, npr. konvoluciju ili množenje matrica), atributa (fiksni parametri te operacije, poput veličine kernela ili koraka), inicijalizatora (konstantne vrednosti ugrađene direktno u graf) i grana (povezuju čvorove i opisuju kojim redosledom podaci teku kroz mrežu), sve do izlaza (opet tip i oblik) [6].

Ovakav graf je acikličan, jer ne postoji putanja koja bi praćenjem usmerenih grana vodila nazad u isti čvor. Uz to je i topološki uređen: ispred svakog čvora u grafu nalaze se svi čvorovi čiji su rezultati potrebni da bi se taj čvor mogao izračunati, čime je zagarantovano da postoji bar jedan validan redosled izvršavanja celog grafa od ulaza ka izlazu.

A. Izvoz modela u ONNX format, provera izvoza i optimizacija grafa

Nakon što je mreža istrenirana model je izvezen iz PyTorch-a u ONNX format radi dalje kvantizacije i poređenja performansi [7].

Prvo je proverena struktura fajla, tj. da li je zapisani graf u skladu sa ONNX specifikacijom, da li se oblici i tipovi svih tenzora mogu izračunati kroz ceo graf i da li su operacije poređane u ispravnom redosledu izvršavanja. Ovo je čisto formalna provera, koja ne govori ništa o tome da li model zapravo daje ispravne rezultate, samo da je fajl validno sastavljen.

Drugo, provereno je da li izvezeni model daje iste rezultate kao originalni PyTorch model. To je urađeno tako što je nekoliko nasumičnih ulaza, različitih veličina *batch*-a, propušteno paralelno kroz originalnu PyTorch mrežu (učitanu sa sačuvanim istreniranim težinama) i kroz izvezeni ONNX model pokrenut u ONNX Runtime-u, nakon čega su njihovi izlazi upoređeni po najvećem apsolutnom odstupanju. Očekivanje je da je ta razlika zanemarljivo mala, jer bi svaka veća razlika značila da je nešto u procesu izvoza pogrešno prevedeno i da model posle konverzije više ne računa isto što i original.

Treće, proverena je kompatibilnost okruženja, tj. kojom verzijom ONNX operator-skupa je model izvezen i da li tu verziju podržava lokalno instalirana biblioteka za izvršavanje ONNX modela. Ovo je provereno tako što je pokušano stvarno učitavanje i otvaranje modela u okruženju za izvršavanje.

Sve tri provere ispravnosti izvoza su uspešno prošle. Izvezeni ONNX model je strukturno validan po ONNX specifikaciji, njegovi izlazi se poklapaju sa izlazima originalnog PyTorch modela uz zanemarljivo odstupanje (najviše 1.8×10^{-6}), a lokalno okruženje za izvršavanje ONNX modela podržava opset verziju 21 sa kojom je model izvezen.

Pre kvantizacije, izvezeni model je dodatno pripremljen skriptom za predobradu: nad grafom je pokrenuta ONNX *shape inference* (upisivanje oblika i tipova svih međurezultata) i optimizacija grafa u ONNX Runtime-u.

Na osnovu svih provera, zaključak je da je izvoz veran originalnoj mreži, čime je izvezeni model potvrđen kao ispravna FP32 referenca za sve dalje korake kvantizacije.

V. KVANTIZACIJA MODELA

Umesto jednog jedinstvenog postupka, u radu je kvantizacija sprovedena kroz tri varijante koje postepeno obuhvataju sve veći deo modela: prva kvantizuje samo težine, druga dodaje i bias vrednosti, a treća uključuje i aktivacije, odnosno međurezultate nastale tokom samog izvršavanja mreže.

Date su i matematičke formulacije ovog postupka [1].

A. Kvantizacija težina

Prva varijanta kvantizacije obuhvata naučene težine konvolucionih i potpuno povezanih slojeva mreže, dok bias vrednosti i sve aktivacije tokom izvršavanja ostaju u FP32 preciznosti.

Kako gotove funkcije za kvantizaciju u korišćenoj biblioteci ne podržavaju ovakav selektivan pristup, ova varijanta je urađena ručnom izmenom ONNX grafa modela.

Za svaki sloj čije se težine kvantizuju, iz grafa se izdvaja odgovarajući tenzor težina W i nad njim se primenjuje simetrična kvantizacija po kanalu. To znači da se za svaki izlazni kanal c , odnosno svaki filter, zasebno određuje faktor skaliranja na osnovu najveće apsolutne vrednosti unutar tog kanala:

$$scale_c = \frac{\max(|W_c|)}{q_{max}} \quad (1)$$

dok je referentna, nulta vrednost (zero point) fiksirana na nulu za sve kanale.

Ovakav pristup po kanalu, umesto jedinstvenog faktora skaliranja za ceo tenzor, opravdan je time što različiti filteri unutar istog sloja mogu imati različite opsege vrednosti.

Na osnovu izračunatih faktora skaliranja, originalne FP32 vrednosti težina se zaokružuju i ograničavaju na opseg celobrojnog tipa:

$$q_c = clip\left(round\left(\frac{W_c}{scale_c}\right), -q_{max}, q_{max}\right) \quad (2)$$

Čime se dobija kvantizovana reprezentacija zapisana kao INT8 ili INT16 ($q_{max} = 127$, odnosno $q_{max} = 32767$), u zavisnosti od varijante koja se generiše.

Originalni FP32 inicijalizator u grafu se zatim zamenjuje ovom kvantizovanim verzijom, zajedno sa pripadajućim faktorima skaliranja, a u graf se dodaje operacija koja pri svakom izvršavanju modela vraća kvantizovane vrednosti nazad u punu preciznost pre nego što uđu u dalji proračun:

$$W_c \approx q_c \cdot scale_c \quad (3)$$

Na ovaj način se ne menja numerički tok izvršavanja modela niti se zahteva podrška za celobrojnu aritmetiku pri inferenciji jedina razlika je u tome kako su parametri zapisani na disku.

B. Kvantizacija težina i biasa

Druga varijanta predstavlja proširenje prve tako što se, pored težina, kvantizuju i bias vrednosti svih slojeva, dok aktivacije i dalje ostaju u punoj FP32 preciznosti tokom izvršavanja.

Postupak je, kao i u prvoj varijanti, sproveden ručnom izmenom ONNX grafa, s tim da je za bias vrednosti primenjen drugačiji pristup nego za težine. Umesto kvantizacije po kanalu, bias b svakog sloja, kvantizuje se kao ceo tenzor, sa jednim zajedničkim faktorom skaliranja:

$$scale = \frac{\max(|b|)}{q_{max}} \quad (4)$$

$$q = clip\left(round\left(\frac{b}{scale}\right), -q_{max}, q_{max}\right) \quad (5)$$

Gde se, za razliku od težina, maksimum apsolutne vrednosti računa preko *svih* elemenata bias vektora odjednom, a ne po pojedinačnim kanalima, jer je bias već 1-D vektor.

Kao i kod prve varijante, kvantizovane vrednosti težina i bias-a se u grafu zamenjuju odgovarajućim celobrojnim zapisom praćenim operacijom koja ih pri svakom izvršavanju modela vraća u punu preciznost.

Postupak je primenjen na svih pet slojeva mreže, a generisane su, kao i ranije, dve verzije modela (INT16 i INT8).

C. Kvantizacija težina, biasa i aktivacija

Treća varijanta obuhvata kvantizaciju celokupnog modela. Pored težina i bias vrednosti, ovde se kvantizuju i aktivacije, odnosno međurezultati koji nastaju tokom samog izvršavanja mreže nad ulaznim podacima.

U odnosu na prve dve varijante, ovde se opseg vrednosti aktivacija ne može unapred izračunati direktno iz sačuvanog modela, jer on zavisi od konkretnih ulaznih podataka koji kroz mrežu prolaze tokom izvršavanja.

Zbog toga ova varijanta više nije urađena ručnom izmenom grafa, kao prethodne dve, već korišćenjem gotove funkcije iz biblioteke za rad sa ONNX modelima koja je namenjena ovakvom vidu kvantizacije [1].

Preduslov za njenu primenu je korak koji se naziva kalibracija, u kom se kroz mrežu propusti reprezentativan uzorak stvarnih podataka, a na osnovu vrednosti koje se pritom pojave na svakom mestu u grafu procenjuje se opseg (minimalna i maksimalna vrednost) koji te aktivacije imaju.

Ta procena služi za određivanje faktora skaliranja kojim će aktivacije biti kvantizovane pri stvarnom izvršavanju modela.

Podešavanja kvantizacije prate isti princip kao u prethodnim varijantama tamo gde je to moguće: težine se i dalje kvantizuju simetrično i po kanalu, sa referentnom nultom vrednošću fiksiranom na nulu. Aktivacije se, kvantizuju asimetrično i po celom tenzoru. Procena opsega aktivacija tokom kalibracije urađena je minimum-maksimum metodom.

Kod kvantizacije samo težina i bias vrednosti bilo je dovoljno u graf ubaciti operaciju koja kvantizovane parametre vraća u punu preciznost, budući da su ti parametri fiksni i unapred poznati.

Kod aktivacija se pored operacije za vraćanje u punu preciznost mora ubaciti i operacija koja te vrednosti kvantizuje u hodu, u realnom vremenu izvršavanja modela. Graf treće varijante stoga sadrži oba tipa operacija naizmenično, za razliku od prve dve varijante u kojima se pojavljuje samo operacija za vraćanje u punu preciznost.

D. Dobijeni grafovi

U odnosu na graf originalne mreže, u verziji gde su kvantizovane samo težine, ispred svakog čvora koji obavlja konvoluciju i svakog čvora koji predstavlja potpuno povezani sloj, dodat je po jedan *DequantizeLinear* čvor [8] koji dekvantizuje odgovarajuću težinu (5 ukupno, po jedan za svaki težinski tenzor). Skala i nulta tačka su nizovi dužine jednake broju izlaznih kanala, znači kvantizacija je po kanalu, ne po celom tenzoru kao što je ranije napomenuto. Bias ostaje netaknut u FP32, kao i Relu, MaxPool i sam račun operacija ostaju identični originalu, tj. i dalje se izvršavaju u FP32. Kvantizacija ovde utiče samo na to u kom obliku su težine sačuvane na disku dok se pre svake upotrebe vraćaju u FP32.

U drugoj verziji gde se dodatno kvantizuju i biasi, oni sada prolaze kroz sopstveni *DequantizeLinear* čvor (dodatnih 5, ukupno 10 u grafu).

Treća verzija podrazumeva parove *QuantizeLinear/DequantizeLinear* [9] umetnute oko svake operacije, sa parametrima dobijenim kalibracijom na reprezentativnom skupu podataka. ReLU čvorovi potpuno nestaju iz grafa, njihov efekat je u *QuantizeLinear* koraku. Ovo je i dalje nefuzionisan graf i sav račun se, uprkos svim ovim Q/DQ čvorovima, i dalje odvija u FP32 između njih.

Kada ONNX Runtime učita prethodni graf i primeni optimizaciju, svi QDQ parovi se sažimaju u par čvorova: jedan *QuantizeLinear* na samom početku (kvantizuje ulaznu sliku) i jedan *DequantizeLinear* na samom kraju (vraća izlaz u float). Svaki konvolucionni sloj zajedno sa svojim QDQ čvorovima stapa se u jedan *QLinearConv* čvor, a svaki potpuno povezani sloj sa svojim QDQ čvorovima u *QGemm*. To su pravi celobrojni kerneli koji množe INT8 vrednosti direktno, bez prolaska kroz float aritmetiku. MaxPool dobija svoju INT8 varijantu, a pojavljuju se i čvorovi za transformaciju tenzora. Ovo je jedina varijanta u kojoj se tokom cele mreže (osim samog ulaza i izlaza) realno računa u celobrojnoj aritmetici.

VI. POREĐENJE MODELA

Modeli se porede po tri kriterijuma: veličini, tačnosti i brzini zaključivanja. Nakon dobijenih prvih rezultata, izvršena je njihova dodatna analiza.

A. Tačnost modela

Tačnost svake varijante modela merena je kao TOP-1 tačnost na celom MNIST test skupu od 10.000 slika.

Tabela VI.1. prikazuje tačnost svih sedam varijanti modela (FP32 osnovni model i po tri kvantizovane varijante u INT16 i INT8 preciznosti).

Tabela VI.1 Tačnost modela

Model	Tip	Tačnost
Base	FP32	0.9891
V1	INT16	0.9891
V2		0.9891
V3		0.9891
V1	INT8	0.9890
V2		0.9889
V3		0.9892

Tačnost je kod svih varijanti praktično nepromenjena u odnosu na FP32 model.

Kod INT16 varijanti nema nijedne promenjene predikcije (0 od 10.000).

Kod INT8 modela, ukupna razlika je 1-2 primera od njih 10.000. S tim da je broj primera u kojima se predikcija stvarno promenila veći (4-9, zavisno od varijante), ali se otprilike podjednako dele na nove greške i slučajne ispravke, pa se njihov neto efekat na tačnost skoro poništi, što prikazuje Tabela VI.2.

Tabela VI.2 Promena predikcija modela sa INT8 kvantizacijom

Model	Broj promenjenih predikcija	Nove greške	Ispravljeno	Neto
V1	5	3	2	-1
V2	4	3	1	-2
V3	9	4	5	+1

B. Veličina modela

Tabela VI.3. prikazuje veličinu modela u KB, kao i smanjenje u odnosu na originalni FP32 model.

Odstupanje ostvarene kompresije od idealnih teorijskih $4\times$ (INT8) i $2\times$ (INT16) ima dva uzroka: delove fajla koji se uopšte ne kvantizuju (struktura grafa, a kod varijante V1 i bias vrednosti koje ostaju u FP32) i nove metapodatke (skale, vrednost nulte tačke) koje kvantizacija dodaje. Kod većih modela ovaj fiksni trošak bio bi zanemarljiv u odnosu na broj parametara, pa bi odnos veličina bio bliži teorijski očekivanom.

Tabela VI.3 Veličine kvantizovanih modela i smanjenje u odnosu na originalni model

Model	Tip	Veličina (KB)	Smanjenje (x)
Base	FP32	255.3 KB	-
V1	INT16	137.7 KB	1.85
V2		138.2 KB	1.85
V3		140.6 KB	1.82
V1	INT8	77.4 KB	3.30
V2		77.7 KB	3.29
V3		80.6 KB	3.17

Kao potvrda ove pretpostavke, može se proveriti koliko memorijskog prostora zauzimaju samo podaci za kvantizaciju, pri čemu je ostatak prostora rezervisan za pomenutu strukturu grafa. Izračunato je smanjenje modela u odnosu na te parametre i dato je u Tabeli VI.4., gde su dobijene vrednosti bliže teorijski očekivanim.

Tabela VI.4 Veličina modela i smanjenje same kvantizovane komponente (bez strukture grafa)

Model	Tip	Ukupna veličina (KB)	Podaci za kvantizaciju (KB)	Smanjenje (x)
Base	FP32	255.27	241.05	-
V1	INT16	137.71	122.37	1.97
V2		138.23	121.94	1.98
V3		140.460	124.25	1.94
V1	INT8	77.44	62.11	3.88
V2		77.73	61.44	3.92
V3		80.57	63.98	3.77

C. Brzina zaključivanja

Brzina zaključivanja merena je na CPU (Intel Core i5-1235U), za batch veličine 1, uz 50 „zagrevajućih“ prolaza koji se ne uračunavaju u merenje i 1000 ponavljanja.

Tabela VI.5. prikazuje izmerena vremena jedne inferencije (u milisekundama, batch = 1) za sve varijante.

Tabela VI.5 Vreme inferencije (ms), batch = 1

Model	Tip	min	median	mean	std
Base	FP32	0.0981	0.1153	0.1970	0.3971
V1	INT16	0.1494	0.1922	0.3118	0.6033
V2		0.1113	0.1253	0.1864	0.3711
V3		0.1457	0.1648	0.4220	1.8344
V1	INT8	0.1499	0.1959	0.2840	0.3790
V2		0.1620	0.2256	0.4454	1.0731
V3		0.1461	0.1612	0.2770	0.8180

FP32 model je najbrži, što je za varijante V1 i V2 očekivano po samoj konstrukciji, jer proračun i dalje teče u FP32, a pri svakoj inferenciji se dodatno izvršava dekvantizacija težina pa ove varijante ne mogu biti brže od osnovnog modela.

Varijanta V3 sa INT8 aktivacijama mogla je da iskoristi celobrojne kernele, ali je model premali pa trošak dodatnih Q/DQ čvorova poništi eventualnu dobit, dok za INT16 aktivacije ONNX Runtime na CPU nema ni celobrojni put izvršavanja.

Razlike između samih kvantizovanih varijanti su u granicama mernog šuma: standardna devijacija je uporediva sa srednjom vrednošću ili veća od nje, pa su min i medijana pouzdaniji pokazatelji od proseka.

Stvarno ubrzanje od INT8 kvantizacije zahtevalo bi hardver sa namenskom podrškom za celobrojnu aritmetiku i model dovoljno velik da ta podrška dođe do izražaja.

Isprobana su i veće vrednosti za batch, ali bez uspeha.

VII. PERSPEKTIVE ZA 4 I 2 BITA

Za konvolucionu neuronsku mrežu izvezenu u ONNX format četvorobitna kvantizacija predstavlja praktično izvodljiv korak. Sam postupak kvantizacije sprovodi se pomoću klase *MatMulNBitsQuantizer* iz modula *onnxruntime.quantization.matmul_nbits_quantizer*, uz mogućnost izbora algoritma kvantizacije: RTN (round-to-nearest) kao podrazumevani, brži ali manje precizan metod, ili GPTQ i HQQ kao tačnije alternative [10].

Kvantizacija se sprovodi po principu blokovske granularnosti, gde parametar veličine bloka, koji mora predstavljati stepen broja dva i biti veći ili jednak 16, određuje segment težina unutar kog se primenjuje zajednički parametar skale i nulte vrednosti. Postoji zahtev za opset verzijom 21 ili višom.

Problem sa ovim pristupom je činjenica da operatorska podrška za blokovsku INT4 kvantizaciju u ONNX Runtime-u ne uključuje operaciju konvolucije u listu podržanih tipova čvorova. Četvorobitna kvantizacija bi se mogla primeniti na potpuno povezane slojeve na kraju mreže, dok bi konvolucionni deo arhitekture zadržao višu numeričku preciznost, osim ukoliko se ne primeni eksperimentalna ili prilagođena implementacija operatora.

Za razliku od četvorobitne kvantizacije, dvobitna (INT2) kvantizacija dostupna je samo u QOperator formatu. Varijante V1, V2 i V3 u ovom radu koriste QDQ format, koji graf gradi od standardnih operatora iz zvaničnog ONNX domena. Pošto su ovi operatori deo samog ONNX standarda, ovako zapisan model može da izvrši bilo koji ONNX-kompatibilan alat za izvršavanje, ne samo ONNX Runtime, čime se zadržava prenosivost i čitljivost grafa.

QOperator nije deo zvaničnog ONNX standarda, već pripada posebnom, proizvođačkom (com.microsoft) domenu, poznatom kao contrib operator [11]. Umesto da graf sadrži zaseban DequantizeLinear čvor praćen operacijom množenja, ceo postupak sakriven je unutar jednog vlasničkog čvora. Posledica toga je da ovakav model može da izvrši isključivo ONNX Runtime, odnosno bilo koji drugi mehanizam za izvršavanje koji je posebno implementirao taj isti operator, čime se gubi prenosivost koju standardni QDQ format poseduje.

A. Očekivana veličina, tačnost i brzina modela pri hipotetičkoj INT4 i INT2 kvantizaciji

Pošto se INT4/INT2 kvantizacija primenjuje na potpuno povezane slojeve, dok konvolucionni slojevi ostaju u FP32, značaj ovog ograničenja zavisi od uдела FC slojeva u ukupnoj memoriji modela. Kod LeNet5 arhitekture taj udeo je dominantan, pa bi teorijska ušteda u veličini modela, iako formalno ograničena samo na deo mreže, u praksi obuhvatila skoro celokupan memorijski prostor.

Teorijski maksimum smanjenja, posmatrano isključivo na nivou sirovih podataka, iznosi 8× za INT4 (sa

32 na 4 bita) i $16\times$ za INT2 (sa 32 na 2 bita) u odnosu na FP32. Taj idealan odnos se, međutim, u praksi ne dostiže u potpunosti, iz istih razloga koji su već uočeni kod INT8/INT16 varijanti u ovom radu.

Dodatno, deo modela koji ostaje u FP32 (konvolucionni slojevi) predstavlja donju granicu ispod koje se ukupna veličina ne može spustiti. Iz ovih razloga se realno ostvareno smanjenje očekuje osetno niže od teorijskog maksimuma, po istom obrascu uočenom kod INT8 varijante (teorijskih $4\times$, ostvarenih $\approx 3.2\text{--}3.3\times$), pri čemu bi taj jaz kod INT2 bio relativno izraženiji nego kod INT4, jer dodatni parametri kvantizacije čine srazmerno veći deo sve manjeg broja bita po vrednosti.

Kada je reč o tačnosti, očekuje se da ona opada sa smanjenjem broja bita, i to ubrzano na nižim preciznostima. Kod INT4 kvantizacije, koja i dalje raspolaže sa 16 diskretnih nivoa po vrednosti, degradacija tačnosti se po pravilu zadržava na niskom nivou, naročito kod jednostavnijih, preparametrizovanih modela kao što je LeNet5 na MNIST skupu. Slično ponašanje je već potvrđeno kod INT8 varijanti u ovom radu, gde je gubitak tačnosti bio zanemarljiv. Kod INT2 kvantizacije, očekuje se znatno izraženiji gubitak tačnosti, posebno ako se koristi jednostavan RTN (*round-to-nearest*) algoritam bez kalibracije. Ovo je i osnovni razlog zbog kog se za ovako nizak broj bita u praksi preporučuju napredniji algoritmi kvantizacije, poput GPTQ i HQQ, koji pokušavaju da nadoknade deo te degradacije.

Ubrzanje izvršavanja kod modela veličine LeNet5, gde je i najveći sloj relativno mali, bi imalo isti obrazac koji je već potvrđen kod INT8 kvantizacije u ovom radu, gde je izmereno usporenje umesto ubrzanja. Ne očekuje se ubrzanje izvršavanja ni kod INT4, a još manje kod INT2 kvantizacije primenjene na model ovakve veličine, već, naprotiv, dodatno usporenje u odnosu na FP32 izvršavanje.

VIII. ZAKLJUČAK

U radu je istrenirana mreža LeNet-5 do tačnosti od 98.91% na MNIST test skupu, izvezena u ONNX format i formalno potvrđena kao verna FP32 referenca (najveće odstupanje u odnosu na PyTorch izlaz reda 10^{-6}). Nad tom referencom sprovedene su i upoređene tri varijante *post-training* kvantizacije koje postepeno obuhvataju sve veći deo modela: samo težine (ručna izmena grafa), težine i bias (ručna izmena grafa) i težine, bias i aktivacije (automatizovano uz kalibraciju).

Rezultati pokazuju da se veličina modela može smanjiti na oko 30% originalne FP32 veličine kod INT8 i oko 54% kod INT16 varijanti, uz tačnost koja ostaje praktično nepromenjena u sva tri slučaja. Kod mreže ove veličine kvantizacija, međutim, ne donosi ubrzanje izvršavanja na CPU: dekvantizacija pri svakoj inferenciji dodaje trošak kod varijanti V1 i V2, dok je kod varijante V3 model premali da bi se iskoristila prednost celobrojnih kernela.

Zaključak je da je kvantizacija, čak i u ovako jednostavnom obliku, pouzdan način da se prostor koji model zauzima na disku smanji bez gubitka tačnosti, dok bi merljivo ubrzanje izvršavanja zahtevalo veći model i hardver sa namenskom podrškom za celobrojnu aritmetiku. Dalji rad mogao bi obuhvatiti primenu na dubljim arhitekturama i merenje na stvarnom ciljnom (*edge*) uređaju.

LITERATURA

- [1] ONNX Runtime, "Quantize ONNX Models," ONNX Runtime Documentation. Dostupno: <https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html>
- [2] CVDF, "MNIST dataset mirror," GitHub. Dostupno: <https://github.com/cvdfoundation/mnist>
- [3] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
- [4] J. Zhang, X. Yu, X. Lei, and C. Wu, "A novel deep LeNet-5 convolutional neural network model for image recognition," *Computer Science and Information Systems*, vol. 19, no. 3, pp. 1463–1480, 2022, doi: 10.2298/CSIS220120036Z...
- [5] PyTorch, "examples/mnist/main.py." Dostupno: <https://github.com/pytorch/examples/blob/main/mnist/main.py>
- [6] ONNX, "IR.md," ONNX GitHub Repository. Dostupno: <https://github.com/onnx/onnx/blob/main/docs/IR.md>
- [7] PyTorch, "torch.onnx," PyTorch Documentation, v2.14. Dostupno: <https://docs.pytorch.org/docs/2.14/onnx.html>
- [8] ONNX, "DequantizeLinear," ONNX Operators Documentation. Dostupno: https://onnx.ai/onnx/operators/onnx_DequantizeLinear.html
- [9] ONNX, "QuantizeLinear," ONNX Operators Documentation. Dostupno: https://onnx.ai/onnx/operators/onnx_QuantizeLinear.html
- [10] ONNX Runtime, "Quantize to INT4/UINT4," ONNX Runtime Documentation. Dostupno: <https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html#quantize-to-int4uint4>
- [11] Microsoft, ONNX Runtime GitHub Repository. Dostupno: <https://github.com/microsoft/onnxruntime/blob/main/docs/ContribOperators.md#commicrosoftmatmulnbits>